Question	Answer
How tanstack query is different from react query? Could you please brief about a bit tanstack-query vs react-query?	They are not two different libraries. React Query was renamed to TanStack Query. React Query (v3, npm react-query) is the old name. TanStack Query (v4+) is the same project. For React you install @tanstack/react-query. The rename happened so the same core could ship for Vue, Solid, Svelte, and Angular — not only React. Caching, refetch, retries, and stale/fresh behavior are the same idea. Do not install both packages in one app; use @tanstack/react-query.
For an application use case where the processing take more than 5 mins (some complex taxk to be done) .. still streaming would be the ux technique or any other design approach altogether?	Use a job, not a live stream as the main UX. Streaming is for “stay here while tokens/progress show up” — usually seconds, maybe a couple of minutes. After ~5 minutes people leave, tabs die, and HTTP streams time out. Better pattern: start the work, return a job id, show status (queued / running / done / failed), let them go elsewhere, then notify when it’s done. Save the result so they can open it later. Streaming is still useful inside that job — a live log, percent complete, or a growing draft — but the source of truth is the saved job, not an open browser connection.
Whats the difference between Websocket and SSE?	SSE — server talks to the browser only, over normal HTTP. Simple, auto-reconnects, text. User still sends via a regular POST. Fits AI token streaming. WebSocket — both sides talk anytime on one open pipe. Text or binary. You handle reconnect. Fits live two-way chat, collab, games. If the user hits Send and the model streams back, SSE is enough. Use WebSocket when the client also needs to keep sending live events on the same connection

Is SSE connection less (webhook) concept?	No. They both “push events,” but they are not the same idea. Webhook: no open connection. When something happens, your server POSTs once to a URL you registered (usually server → server). Fire and forget (plus retries). SSE: keeps a connection open. The browser connects and stays on the line; the server writes events down that pipe. Not connection-less.
how come OrderMitra is unidirectional? user also submit queries and susequent interactions rt?	The chat is two-way. The SSE pipe is one-way. What you do: You type and hit send → that is a normal HTTP POST (user → server). The model answers → that reply streams down SSE (server → you). Next question → another POST. Stream again.
In eventSource, how will we know when to close the connection (source.close()), how will it know when the server has finished sending data	EventSource does not treat “stream ended” as done. If the server just closes the HTTP response, the browser fires error and reconnects. You have to close it yourself. How you know it’s finished: an app signal in the stream, not TCP going quiet. Typical pattern: Server sends a last event, e.g. event: done or data: [DONE] Client handler sees that → source.close() Until you call close(), readyState is not CLOSED, so it will try to come back. That’s why many chat UIs (including the Vercel AI SDK style) use fetch + readable stream instead of EventSource: when the response body ends, the stream is over. No auto-reconnect. EventSource is built for a feed that stays open (news, ticks). One-shot “answer this prompt” fits fetch streaming better.

how come OrderMitra is unidirectional and use SSE not WebSocket? user also submit queries and susequent interactions rt?	The chat is two-way. The SSE pipe is one-way. What you do: You type and hit send → that is a normal HTTP POST (user → server). The model answers → that reply streams down SSE (server → you). Next question → another POST. Stream again.

Does WebSockets handle the coding required for a Publish-Subscribe (Pub/Sub) architecture out of the box? Could you provide a code example to demonstrate this?	No. A WebSocket is only a two-way pipe: send bytes, receive bytes. It does not know topics, subscribers, or fan-out.
	
	Pub/sub is your (or a broker's) layer on top: subscribe to a topic, publish to that topic, server copies the message to everyone on the list. Redis, Kafka, MQTT, Socket.IO rooms, Ably, etc. do that. Raw WebSocket does not. const topics = new Map(); // topic -> Set of sockets
	
	function subscribe(ws, topic) {
	if (!topics.has(topic)) topics.set(topic, new Set());
	topics.get(topic).add(ws);
	}
	
	function publish(topic, payload) {
	const msg = JSON.stringify({ topic, payload });
	for (const client of topics.get(topic) ?? []) {
	if (client.readyState === WebSocket.OPEN) client.send(msg);
	}
	}
	
	wss.on("connection", (ws) => {
	ws.on("message", (raw) => {
	const { action, topic, payload } = JSON.parse(raw);
	if (action === "subscribe") subscribe(ws, topic);
	if (action === "publish") publish(topic, payload);
	});
	ws.on("close", () => {
	for (const set of topics.values()) set.d

So, basically, we need SSE for streaming part?	For the streaming part of OrderMitra-style chat: yes. Server → UI tokens/progress. SSE (or fetch that reads an SSE-shaped body) is the usual fit. You still use a normal POST to send the user's message. WebSocket is optional, not required for that pattern.

If i understand correcly... Any AI assistant (like Chatgpt, claude etc) text based chatbot uses SSE where the response tokens are streamed to the user. Whereas the voice based chat uses Web sockets because I will be talking to the AI assistant which needs both way communication. Is that understanding correct ?	Text vs voice: Your idea is mostly right, with one fix. Text chat (ChatGPT, Claude): you POST the message; tokens come back as a stream (SSE or fetch streaming). The chat is still two-way; only the token pipe is one-way. Voice: you need continuous two-way audio (talk, listen, interrupt). SSE is a poor fit (text, one direction, not audio frames). Voice uses WebSocket and/or WebRTC. WebRTC is common for low-delay audio. Some APIs (e.g. OpenAI Realtime) use a WebSocket for events + audio. Short version: text ≈ SSE for the reply. Voice ≈ duplex socket/media, not SSE.
Why did not we used websockets for SSE in our first demo?	The first demo is ask a question, watch the answer type out. That only needs server → screen for tokens. You already send the question with a normal POST. WebSockets would work, but you'd take on extra work (connect, reconnect, two-way protocol) for no extra benefit. SSE (or fetch streaming, same idea) matches that: one-way stream, simple, enough for ChatGPT-style text. Save WebSockets for live two-way stuff (voice, many clients talking at once).

IN this casem how do we do give the system prompt and do prompt engineering?	In Vercel AI SDK you pass a system string (and the chat messages) into streamText / generateText. That’s your prompt engineering slot. Typical route: streamText({ model: groq("llama-3.3-70b-versatile"), system: `You are OrderMitra. Answer only from tool results or the order data given. If you don't know, say so. Be short. Don't invent order IDs.`, messages, // from useChat tools: { /* getOrder, searchOrders */ }, }); What to put in system: who it is, what it can do, what it must not do, output shape (short, bullets, citations), and “use tools / given data only.” What not to stuff there: the whole database. Fresh facts go in retrieved context or tool results each turn. system is the standing rules; it changes rarely. On the client, useChat just sends user messages. Prompt work lives on the server so users can’t rewrite your system prompt.

Then how can we ground the LLM response with our data if needed with Vercel AI SDK?	You don’t “turn on grounding” in the SDK. You put your data in the loop before or during the stream. Three usual ways with Vercel AI SDK: 1. Stuff context in the prompt (simple RAG) On each request: fetch relevant rows (search, SQL, embeddings), then send them in system or as extra messages: “Here are the matching orders: … Answer only from this.” Stream as usual (streamText). Good for small, already-fetched snippets. 2. Tools (what OrderMitra-style apps do) Give the model tools like getOrder, searchOrders. The model calls them, your code hits Prisma/DB, results go back, then it answers. Use streamText / useChat with tools and multiple steps so it can look up, then talk. The stream can include tool events, then the final text. 3. Hybrid Retrieve a short list first (or via a search tool), then generate. Don’t dump the whole DB into the prompt. Rules that actually ground it: retrieve only what’s needed, tell the model to stick to that data, and show citations (“Order #123”) so you can check it. The SDK str

How does the frontend handle different events like tool call starting, end, HITL etc. The frontend has to understand these different events and render different components accordingly. Is that abstracted by Vercel AI SDK ?	Mostly yes — if you use the SDK on the client (useChat), you don't parse SSE yourself. It turns the stream into messages with parts. You still choose the UI for each part. What the SDK gives you: A growing messages list Each message has parts: text, tool call in progress, tool result, maybe a "waiting for you" approval Status like streaming / ready / error What you still write: If part is text → chat bubble If tool started → "Looking up order…" spinner If tool finished → show the result card If HITL (human-in-the-loop) → Yes/No or a form, then send the approval back You do not have to handle tool-call-start bytes by hand. You do have to map over parts and render the right component. HITL is not magic UI. The SDK can pause the agent (tool needs approval). Your frontend shows the confirm button and calls the SDK helper to continue. If you skip useChat and read the stream yourself, that abstraction is gone — you parse events.

why is generateObject and streamObject showing as deprecated?	They are deprecated because the SDK merged two APIs into one. Old idea: generateText / streamText = prose. generateObject / streamObject = JSON with a schema. The object helpers were weaker (e.g. tools didn't fit as well). New idea: always use generateText / streamText. If you want structured JSON, add output: Output.object({ schema }). They still work (strikethrough / warning). They will be removed later. For class code, prefer: import { streamText, Output } from "ai"; const result = streamText({ model, output: Output.object({ schema: yourZodSchema }), prompt: "...", }); Same job as streamObject — one function instead of two. Slides/labs may still show the old names; that's why you see deprecated.
We can use claude code and point the setup document to it and it installs all the prerequisites	Yes. You can open the project in Claude Code, attach Module_5_Pre_Session_Setup_Guide.pdf, and ask it to follow the steps: npm install, Prisma generate/push/seed, check Node version, start npm run dev.
Zod schema is manadatorily required to use in implementation? Or its users choice during implementation ?	Not mandatory for the whole app. Only when you want structured output (JSON the UI can trust). Chat that streams plain text (streamText with no output) → no Zod. You want a recipe object / order card with fields → you need a schema. Zod is the usual one in these labs. The SDK can also take other schema libs; Zod is a choice, not a law. Think of Zod as a contract: "the model must return this shape." Skip it if you're fine with free text. Use it when the frontend will render fields, not a paragraph.

I am getting error while usingcommand - npx tsx prisma/seed.ts Cannot find module 'C:\user\Documents\\module5_handson_tutorial\node_modules\@prisma\client\runtime\library.js' at createEsmNotFoundErr (node:internal/modules/cjs/loader:1493:15) at finalizeEsmResolution (node:internal/modules/cjs/loader:1482:9) at resolveExports (node:internal/modules/cjs/loader:685:14) at Module._findPath (node:internal/modules/cjs/loader:752:31) at node:internal/modules/cjs/loader:1441:27	The Prisma client wasn't generated (or node_modules is incomplete). seed.ts imports @prisma/client, which expects that library.js file after prisma generate. From the project folder (module5_handson_tutorial): npm install npx prisma generate npx prisma db push npx tsx prisma/seed.ts generate must succeed before seed. If generate errors, paste that output. If it still fails: delete node_modules, run npm install again, then npx prisma generate, then seed. Don't skip generate — that's the usual miss from the setup guide.
so basically every question/reply by user is a post req to backend server and the response from server to User(UI) is through SSE. when is the SSE connection established in app like chatGPT Gemini? When the GET call to establish SSE with server is called from UI? Also if each POST request sends only the text the user entered at that point how the current context or chat history is preserved in backend per user?	Mostly yes for one turn: UI sends the new message (POST). Tokens come back on a stream. In ChatGPT-style apps that stream is usually the same POST's response body (text/event-stream or a chunked body), not a separate "open SSE, then POST." When is the stream opened? When you hit Send, not when you open the home page. Each answer is a new stream. When the model finishes, that stream ends. Next question → new POST + new stream. GET + EventSource? The browser's EventSource is GET only, so labs sometimes show GET. Real chat APIs almost always POST (body can include history + the new line). The UI uses fetch and reads the response as a stream. Same SSE format, different HTTP method. No extra GET required. History if POST only has the new sentence? Two patterns (apps often mix them): Client sends the thread — every POST includes prior messages plus the new one. Backend can be stateless for that call. useChat does this: messages in the request. Server remembers — POST has conversationId + new text. Backend loads

Now getting this error Prisma schema loaded from prisma\schema.prisma. Error: Prisma schema validation - (get-config wasm) Error code: P1012 error: The datasource property `url` is no longer supported in schema files. Move connection URLs for Migrate to `prisma.config.ts` and pass either `adapter` for a direct database connection or `accelerateUrl` for Accelerate to the `PrismaClient` constructor. See https://pris.ly/d/config-datasource and https://pris.ly/d/prisma7-client-config --> prisma\schema.prisma:7 | 6 | provider = "sqlite" 7 | url = "file:./dev.db" | Validation Error Count: 1 [Context: getConfig] Prisma CLI Version : 7.9.1	his is a version mismatch, not a bad SQLite path. The lab schema is old style: url = "file:./dev.db" inside schema.prisma. You ran Prisma CLI 7.9.1, which no longer allows url there. So validation fails. For this class: don’t use Prisma 7. Use the Prisma version in the project (package.json / lockfile). From module5_handson_tutorial: npm install npx prisma generate If it still says CLI Version : 7.9.1, you’re not using the local one. Check package.json for "prisma" / "@prisma/client" (likely 5.x or 6.x). Then: npm install prisma@<that-version> --save-dev npm install @prisma/client@<that-version> npx prisma generate npx prisma db push npx tsx prisma/seed.ts Don’t follow the Prisma 7 “move URL to prisma.config.ts” docs unless the instructor upgraded the lab. That rewrite is extra work and not what OrderMitra was written for. Rule: npx prisma without a pinned project can download latest (7.9). Always install from the repo first, then run Prisma through that install.
what was the name of the open source model which Prof. Mondal recommended?	openAI

Does mean that 'tool' where LLM talks to database configured in project?	n this lab, yes — that's the usual tool. Example: getOrder / searchOrders → your code runs Prisma against the SQLite orders DB → result goes back to the model → it answers in English. A tool is just a function you allow the model to call. It is not always a database. It could be search, calendar, "send email," etc. The LLM does not open the DB itself. It says "call getOrder with id 12." Your server talks to Prisma. That's safer and keeps SQL in your code.
can we have mutiple tools added?	Yes
openai/gpt-oss-120b	What it is: OpenAI's open-weight model gpt-oss-120b (~120B parameters). Weights are public (Apache 2.0). It is not ChatGPT / GPT-4 on OpenAI's paid API. Why it looks like openai/gpt-oss-120b: Groq (and others) host it. The openai/ prefix is Groq's name: "OpenAI's OSS model, 120B." You still use your Groq key. Traffic goes to Groq, not to api.openai.com. In the lab: groq("openai/gpt-oss-120b") means "run this OSS model on Groq." Same streaming/tools pattern as Llama — only the brain behind the API changes. Smaller sibling: openai/gpt-oss-20b (faster/cheaper, weaker).
what is a StepBadge ? a component?	Yes — it's a UI component in the demo, not something Groq or the AI SDK provides. It's a small label/chip (a "badge") that shows which step the agent is on, e.g. "Calling tool," "Step 2," "Looking up order." It reads the SDK message parts / step state and renders a pretty tag so students can see the agent loop, not just the final text. You could name it anything (StepChip, AgentStage). The idea is: map stream events → a visible step marker.

can we call an API with Auth in the tools? if so how do we do?	Yes. A tool is your server function. It can fetch any API. The model only picks when and with what arguments (order id, city). You attach auth.
	
	Secrets (service API key) — stay on the server, in .env.local. Never put them in the system prompt or tool args the model invents.
	
	execute: async ({ orderId }) => {
	const res = await fetch(`https://api.example.com/orders/${orderId}`, {
	headers: { Authorization: `Bearer ${process.env.ORDERS_API_KEY}` },
	});
	return res.json();
	}
	User auth (this logged-in person) — read the session in the route (cookie/JWT), then close over it in the tool. The LLM must not pass the password.
	
	// in POST handler, after you know user
	execute: async ({ orderId }) => {
	return getOrderForUser(orderId, session.userId); // your DB/API checks they own it
	}
	Don’t: let the model fill apiKey or Authorization.
	Do: validate args, check the user is allowed, then call the API.
	
	Dangerous calls (refund, delete) → same tool pattern + HITL (user confirms) before execute runs.

All the AI coding agents (eg: copilot agent) take our permission to execute any command or change any internal project dependencies, is this the same implementation of HITL?	Same idea, different product. Both are human-in-the-loop: the agent proposes a side effect; you approve; then it runs. Copilot/Cursor: “Run npm install?” / “Edit package.json?” → you click Allow. OrderMitra HITL: “Refund this order?” → you click Yes in the chat. That’s the pattern: don’t execute dangerous tools until a human says yes. Not the same code. The IDE has its own approval UI, sandbox, and “always allow.” Your app uses SDK tool approval + your button. Conceptually HITL; not “we reused Copilot’s implementation.” Read-only tools (search orders) often skip HITL. Writes, shell, payments, dependency changes → ask first.

Can we use SSE in Spring boot based microservices architecture?	Yes. SSE is just HTTP: one long response with Content-Type: text/event-stream. Spring can do that. Servlet (typical Spring MVC): SseEmitter — you send events with emitter.send(...), complete when done. WebFlux: return Flux<ServerSentEvent<...>> — nicer if the rest of the stack is reactive. In a microservice setup it still works, with a few ops facts: The stream lives on one instance. The browser is connected to that pod until the reply finishes. Load balancers must not kill idle HTTP** (raise timeout; some proxies buffer — turn buffering off for that route). Next question can hit another instance. That's fine if history is in DB or the client sends the full messages list. Don't put SSE behind a gateway that waits for the full body. It must flush chunks. For many users, each open stream holds a thread/connection — WebFlux scales that better than blocking MVC. Same pattern as the lab: client POST (or GET) to your Spring API → service calls OpenAI → you write tokens on the SSE emitter. Vercel AI SDK is

Sometimes user interaction (HITL) also come after act of agent like “keep” or “undo” code. So how and where can we acheive the same?	There are two HITL moments. Labs often show only the first. Before the act: “May I refund / run this command?” Tool does not run until Yes. That’s SDK needsApproval. After the act: agent already did something (wrote code, updated a draft). You get Keep or Undo. Copilot/Cursor work this way on diffs. How to get Keep/Undo Agent runs the tool (or writes a draft, not production). You save enough to reverse: old file contents, a git commit, a pending row in DB. UI shows the change + Keep / Undo. Keep → mark accepted (commit, set status applied). Undo → restore snapshot / revert commit / delete the draft. Where UI: Keep/Undo buttons (not the same as the pre-approval dialog). Server: store “before” vs “after”; Keep/Undo are normal POSTs (/accept, /revert). The SSE stream can already be finished. Optional: still stream “I edited foo.ts” as a part; the buttons sit on that card. Pre-approval = don’t do harm. Keep/Undo = do it, but make it reversible. For code, git (or a patch + original text) is the usual undo. Fo

can we have code example of HITL after act with tool in sequence?	After-the-act HITL: the tool runs first, you save an undo, then the user hits Keep or Undo. That is a second HTTP call, not needsApproval.
	
	1. Tool applies a draft and stores “before”
	
	// lib/changes.ts
	const pending = new Map<string, { before: string; after: string; path: string }>();
	export async function applyDraft({ path, after }: { path: string; after: string }) {
	const before = await readFile(path, "utf8");
	await writeFile(path, after); // already changed on disk
	const id = crypto.randomUUID();
	pending.set(id, { before, after, path });
	return { changeId: id, path, preview: after.slice(0, 200) };
	}
	export async function keep(changeId: string) {
	pending.delete(changeId); // accept: leave file as-is
	return { ok: true };
	}
	export async function undo(changeId: string) {
	const c = pending.get(changeId);
	if (!c) return { ok: false, error: "unknown or already decided" };
	await writeFile(c.path, c.before);
	pending.delete(changeId);
	return { ok: true };
	}

Suppose there is a data update operation going on via a prompt ("update the user records"). lets say this is updating 100 records. now, in the middle, lets say the user in the client side, clicks stop/abort. the useChat abort sends an abort signal to the backend. but by that time already some records have been updated already. even if the backend aborts now, is there a way to rollback the whole transaction?	Only if those 100 updates are still inside one uncommitted DB transaction. Abort does not magically undo rows you already committed. useChat stop = cancel the HTTP/LLM call. If 40 rows are already committed, they stay. The signal only helps you stop doing more. All-or-nothing: wrap the batch in $transaction (or SQL BEGIN … COMMIT). If abort fires, throw inside the transaction → database rolls back all 100. Works only while nothing has been committed yet. If you commit per row: abort after 40 → 40 remain. To undo you need old values saved and an explicit Undo, or write to a draft table and apply only at the end. So: rollback of the whole prompt = your transaction (or undo log), not the abort button by itself. Pass abortSignal into the tool so you stop early; pair it with one transaction or staging + apply.
what is the plugin/extension being used by the instructor to display .db?	The setup guide's official viewer is Prisma Studio, not a VS Code plugin: npx prisma studio → browser at localhost:5555 → open the Order table. If the instructor clicks a .db file inside the editor, that's usually a SQLite extension. Common ones: SQLite Viewer (Florian Klampfl / qwtel.sqlite-viewer) — click dev.db, see tables SQLite (alexcvzz.vscode-sqlite) Ask them the exact name if it matters for class. For the lab, Prisma Studio is enough

Please give a code example of prisma with PostgreSQL	Same Prisma style as the lab, different database.
	
	.env
	
	DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/order_mitra?schema=public"
	prisma/schema.prisma
	
	generator client {
	provider = "prisma-client-js"
	}
	datasource db {
	provider = "postgresql"
	url = env("DATABASE_URL")
	}
	model Order {
	id Int @id @default(autoincrement())
	customer String
	item String
	status String
	createdAt DateTime @default(now())
	}
	(If you are on Prisma 7, url may live in prisma.config.ts instead — for class, pin Prisma 5/6 like the lab.)
	
	Commands
	
	npx prisma generate
	npx prisma db push
	# or: npx prisma migrate dev --name init
	Use in a route / tool
	
	import { PrismaClient } from "@prisma/client";
	const prisma = new PrismaClient();
	export async function getOrder(id: number) {
	return prisma.order.findUnique({ where: { id } });

prisma support semantic search?	Not as a built-in “semantic search” feature. Prisma is an ORM: SQL tables, findMany, relations. It does not embed text or rank by meaning by itself. Semantic search = turn text into vectors, store them, find nearest neighbors. Typical stack: embeddings API + pgvector (Postgres) or a vector DB (Pinecone, etc.). You can use Prisma with that: Postgres + pgvector extension Prisma Unsupported("vector") or raw SQL ($queryRaw) for ORDER BY embedding <=> $1 Or a Prisma pgvector extension So: Prisma talks to the DB. You create embeddings and run the vector query. For the lab, OrderMitra uses normal filters (customer, status), not semantic search.

How can we prevent exposing all columns to the LLM and only send selected columns using Prisma?	The model only sees what your tool returns. Don’t return prisma.order.findMany(). Pick columns with select (or omit).
	
	execute: async ({ customer }) => {
	return prisma.order.findMany({
	where: { customer: { contains: customer, mode: "insensitive" } },
	select: {
	id: true,
	item: true,
	status: true,
	// no email, phone, card, internal notes
	},
	take: 10,
	});
	}
	Also:
	
	Never pass the raw Prisma object if it has extra fields — select (or map to { id, item, status }).
	Limit rows (take) so you don’t dump the table into the prompt.
	Strip secrets in code even if someone later adds a column to the schema.
	Prisma still can read all columns in the DB. You choose the projection before the result goes to the LLM. That’s the control.

Suppose I want to use Oracle Database for backend. Can you guide which file and setup chnages?	Prisma cannot talk to Oracle. Official providers are SQLite, Postgres, MySQL, SQL Server, MongoDB, etc. There is no provider = "oracle". Don’t try to only swap the URL in schema.prisma — it will not work. For this OrderMitra / Node lab, Oracle means replace Prisma (or keep Prisma off the Oracle path). What you’d change prisma/schema.prisma — stop using it for Oracle (or leave SQLite for the demo only). .env / .env.local — not DATABASE_URL for Prisma. Something like: ORACLE_USER, ORACLE_PASSWORD, ORACLE_CONNECTION_STRING (Easy Connect: host:1521/FREEPDB1). package.json — add oracledb (and Instant Client on the machine). Remove Prisma for that path if you fully switch. prisma/seed.ts — rewrite as SQL/oracledb inserts (or SQL*Loader). Every PrismaClient usage (tools, API routes) — findMany / update become connection.execute(...).

Suppose I want to use Oracle Database for backend. Can you guide which file and setup chnages?	Prisma cannot talk to Oracle. Official providers are SQLite, Postgres, MySQL, SQL Server, MongoDB, etc. There is no provider = "oracle". Don’t try to only swap the URL in schema.prisma — it will not work. For this OrderMitra / Node lab, Oracle means replace Prisma (or keep Prisma off the Oracle path). What you’d change prisma/schema.prisma — stop using it for Oracle (or leave SQLite for the demo only). .env / .env.local — not DATABASE_URL for Prisma. Something like: ORACLE_USER, ORACLE_PASSWORD, ORACLE_CONNECTION_STRING (Easy Connect: host:1521/FREEPDB1). package.json — add oracledb (and Instant Client on the machine). Remove Prisma for that path if you fully switch. prisma/seed.ts — rewrite as SQL/oracledb inserts (or SQL*Loader). Every PrismaClient usage (tools, API routes) — findMany / update become connection.execute(...).

Is it better practice to have multiple Prisma schemas and functions, or should we keep functions in separate files? I have the same question regarding API routes.	Prisma schema: one schema.prisma per database for this lab and most apps. Split models only when the product is huge (Prisma can use a prisma/ folder with several .prisma files that still make one client). Don't make a new schema per feature. Prisma usage (functions): yes, separate files. One PrismaClient (singleton). Queries live in lib/db/orders.ts (or a small repo). Routes and tools call those functions. Don't paste prisma.order.findMany in every route.ts. API routes: split by URL / use case, not by every helper. /api/chat — streaming chat (one route is enough). /api/orders — REST list/get if you need it. Don't create /api/getOrder, /api/getOrder2 for each function — those are functions, not routes. Rule of thumb: one schema, many small modules, few routes. Routes are HTTP doors. Functions are how you talk to Prisma. Tools import the same functions the routes use so you don't duplicate SQL.
Is there something like vercel for Rails?	Vercel is built around Next.js / frontends. Rails does not have a first-class "Vercel for Rails" from Vercel. You deploy Rails elsewhere. Closest PaaS (git push → URL, like Vercel's feel): Render / Railway / Fly.io — common for small Rails apps Heroku — the old default; still works, costs more Hatchbox — Rails-focused, on your VPS Rails 8's own path: Kamal (Docker on a cheap VPS). That's "official," not serverless like Vercel. Serverless (Vercel-style functions) and Rails don't match well: Rails likes a long-lived app + DB. Use a always-on host + Postgres. If you meant Vercel AI SDK, not hosting: in Ruby you'd use a gem (ruby-openai, ruby-llm, etc.) plus SSE in Rails. Different ecosystem, same ideas (stream, tools).